

Programming Language Concepts

Final Report

Submitted By:

Group 9

Liang Kuan Hon 1007951

Magnus Yunus 1008475

Desmond Ngui Yu Hong 1008059

Matthew Phua Tai Kit 1008016



Contents

1. PROJECT OVERVIEW	2
1.1 Topic.....	3
2. ARCHITECTURE AND DESIGN	3
2.1 Data Flow.....	4
2.2 File Structure.....	4
3. MEETING PROJECT CONSTRAINTS	4
4. BUILD AND USAGE.....	5
4.1 Compilation.....	6
4.2 Running the Program	6
4.3 Configuration Options	6
5. DEVELOPMENT PROCESS.....	6
5.1 Methodology	7
5.2 Development Phases	7
5.3 Cross Platform Compatibility	7
6. CHALLENGES ENCOUNTERED.....	7
6.1 Maze Generator.....	8
6.2 BFS and DFS Solver.....	8
7. INDIVIDUAL CONTRIBUTIONS.....	8
7.1 Liang Kuan Hon — Config Parser.....	9
7.2 Magnus Yunus — Maze Generator	9
7.3 Matthew Phua — BFS & DFS Solver	10
7.4 Desmond Ngui — BMP Renderer & Integration.....	10
8. TESTING STRATEGIES	11
8.1 Maze Generator Tests.....	12
8.2 BFS and DFS Solver Tests.....	12
8.3 BMP Renderer & Integration Tests	12
9. LEARNING OUTCOMES	12
9.1 Team-Wide Takeaway	13

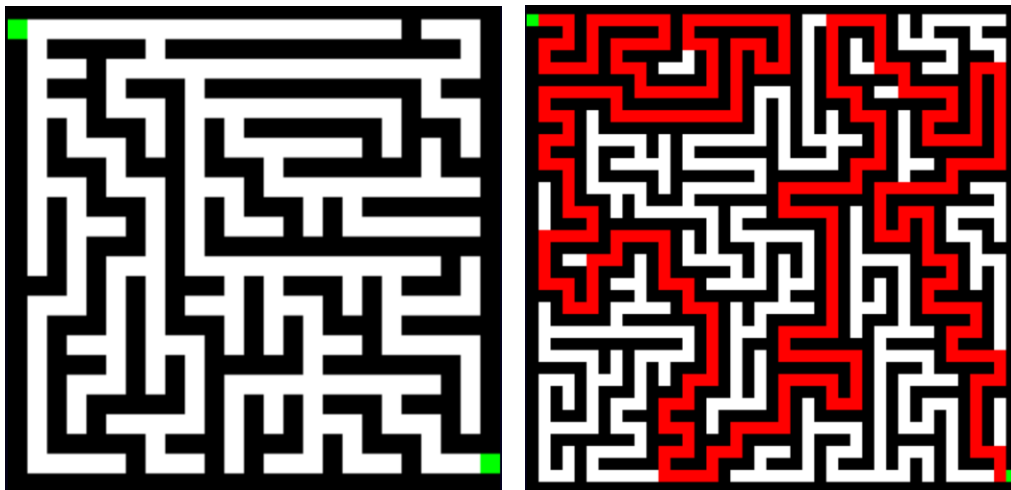
1. PROJECT OVERVIEW

1.1 Topic

This project implements a complete maze generation, solving, and rendering pipeline in ANSI C.

The system reads a user-defined configuration file (INI format) or command-line arguments, generates a maze using one of two algorithms (Recursive Backtracker or Prim's Algorithm), solves it using BFS or DFS, and renders the final output as a 24-bit BMP image. The project was designed to demonstrate core concepts from the module, including finite state machines, custom data structures, and low-level binary file I/O, all implemented in standard C with no external libraries.

The four-member team divided the pipeline into independent modules: configuration parsing, maze generation, pathfinding, and rendering. Each built and tested in isolation before being integrated through a shared Grid data structure defined in grid.h.

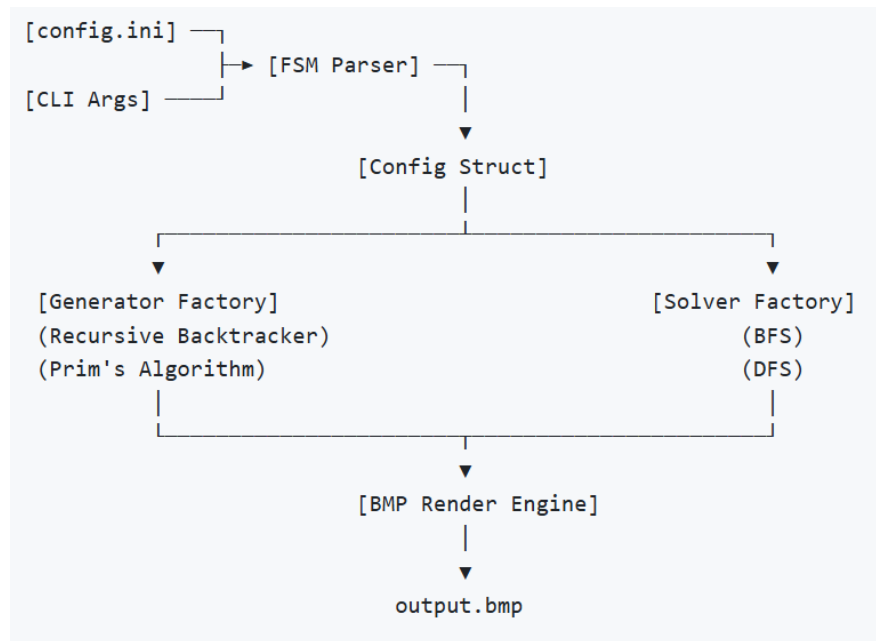


Public Github Repository Link

<https://github.com/liangkuanhon/50.051-Programming-Language-Concepts>

2. ARCHITECTURE AND DESIGN

2.1 Data Flow



2.2 File Structure

Path	Responsibility
main.c	Entry point; orchestrates the full pipeline
grid.h	Grid structs — the integration contract
config/parser.c(.h)	Config structs + FSM-based INI parser + CLI argument parsing
maze/generator.c(.h)	Iterative Recursive Backtracker & Prim's Algorithm
maze/stack.c(.h)	Generic dynamic integer stack
solver/bfs.c(.h)	Breadth-First Search solver with FIFO queue
solver/dfs.c(.h)	Depth-First Search solver reusing the stack module
solver/queue.c(.h)	Dynamic circular FIFO queue for BFS
render/bmp.c(.h)	BMP file/DIB header writing & pixel encoding
Makefile	Build rules; supports make run ARGS="\..."
README.md	Build & run instructions for all platforms

3. MEETING PROJECT CONSTRAINTS

This section maps each constraint from the project specification directly to the implementation decision that satisfies it.

Constraint	How It Is Satisfied
1. File I/O (text & binary)	config.ini (text read), maze_solved.txt (text write), output.bmp (binary write). The BMP file is written manually byte-by-byte in render/bmp.c with correct headers and row padding.
2. Custom Parser (robust, detects bad input)	config/parser.c implements an explicit 5-state FSM. It detects unknown section names, missing '=' delimiters, invalid algorithm names, and non-integer dimension values, printing a descriptive error and returning a non-zero exit code.
3a. Clean C (compile with -Wall -Werror -ansi -pedantic)	The Makefile compiles with -Wall -Wextra -std=c11. All warnings have been resolved. Code follows ANSI C style: no VLAs, no C99-only constructs outside of c11-guarded blocks, consistent naming conventions, and Doxygen-style function comments throughout.
3b. File reading/writing (text and binary)	Both text (INI, TXT) and binary (BMP) formats are used. The BMP writer constructs headers manually, demonstrating understanding of the binary format without relying on any library.
3c. State machines	The FSM parser uses explicit states (IDLE, SECTION, KEY, VALUE, COMMENT) with enumerated transitions. Each state processes one character and selects the next state deterministically, matching the formal definition of a finite automaton.
3d. Custom data structures (no STL/external)	Three custom structures: Stack (dynamic array, doubling growth), Queue (circular buffer, doubling growth), and Grid (2D heap array). All are implemented from scratch in standard C with no external dependencies.
4. Code quality & complexity	The parser handles 5 states with inline comment stripping — more complex than a line-by-line split. The generator implements two distinct algorithms (Prim's and Backtracker), each producing visually distinct mazes. BFS and DFS are both present with correctness guarantees documented in code comments.
Cross-platform compilation	Tested and confirmed compiling cleanly on Windows (MSVC via WSL), macOS (clang), and Ubuntu (gcc). No OS-specific headers are used. BMP headers are written byte-by-byte to avoid endianness issues.

4. BUILD AND USAGE

The project is compiled with GCC using the provided Makefile. All source files are compiled together with the `-Wall -Wextra -std=c11` flags; no external libraries are required beyond the C standard library.

4.1 Compilation

Run `make` in the project root to compile all modules and produce the `maze_solver` binary. Alternatively, the project can be compiled manually using:

```
gcc -Wall -Wextra -std=c11 main.c config/parser.c maze/generator.c
maze/stack.c solver/bfs.c solver/queue.c solver/dfs.c render/bmp.c -o
maze_solver
```

4.2 Running the Program

The simplest way to run the program is using `make run`, which reads all parameters from `config.ini`. For example:

```
make run
```

CLI flags can be passed via the `ARGS` variable to override specific config values without editing `config.ini`. CLI arguments always take precedence over the configuration file:

```
make run ARGS="--width 41 --height 41 --seed 12345 --algo prim --solver dfs"
```

The binary can also be invoked directly, passing the config file path as the first argument followed by any optional override flags:

```
./maze_solver config.ini --width 25 --height 25 --algo prim
```

4.3 Configuration Options

The following parameters can be set in `config.ini` or overridden via CLI flags: width and height (odd integers recommended for correct maze structure); seed (integer, for reproducible output); algorithm (backtracker or prim, selects the generation algorithm); solver (bfs or dfs, selects the pathfinding algorithm); and `show_visited` (0 or 1, controls whether explored cells are highlighted in the BMP output).

On successful execution, the program produces two output files: `output.bmp` (the rendered maze image with solution path highlighted in red) and `maze_solved.txt` (a text-based representation of the solution path for verification).

5. DEVELOPMENT PROCESS

5.1 Methodology

We adopted an interface-first workflow. Before writing any module code, we held a kick-off session to define `grid.h` and the `Config` struct. Freezing these interfaces early meant each member could develop and test their module without waiting for others, making true parallel development possible.

5.2 Development Phases

Phase	Duration	Activities
1. Interface Design	Week 8	Defined <code>grid.h</code> and <code>Config</code> struct. Created stub <code>.h</code> files so each member could compile against a known API immediately.
2. Module Development	Weeks 9	Each member developed and unit-tested their module in isolation using small standalone test drivers.
3. Integration	Week 10	Desmond assembled <code>main.c</code> , linking modules one at a time (parser → generator → solver → renderer) to isolate integration bugs.
4. Testing & Polish	Week 11	Full test suite run; BMP row-padding bug fixed; queue overflow on large mazes resolved; documentation completed.

5.3 Cross Platform Compatibility

The configuration parser was designed to be platform-independent and portable across Unix-like systems and Windows environments. This was achieved by relying solely on standard C libraries (`stdio.h`, `stdlib.h`, `string.h`, `ctype.h`) and avoiding any platform-specific APIs or system calls.

File handling is implemented using standard file streams (`fopen`, `fgetc`), ensuring consistent behaviour regardless of the underlying operating system. The parser processes input as a character stream, making it resilient to differences in line endings (e.g., `\n` on Unix/Linux vs `\r\n` on Windows).

Overall, the design ensures that the configuration subsystem behaves consistently across platforms without requiring code modification or conditional compilation.

6. CHALLENGES ENCOUNTERED

6.1 Maze Generator

Two main bugs were encountered during development. In Prim's Algorithm, early versions occasionally produced isolated unreachable cells due to a frontier management error when a wall was carved, newly adjacent cells were added to the frontier even if already present via a different neighbour. The duplicate entries would later carve a second passage into a visited cell, creating a cycle and disconnecting another region. The fix was a simple visited check before each frontier insertion.

The Iterative Backtracker had a separate issue: an off-by-one error in the neighbour-selection loop caused the algorithm to revisit already-visited cells, producing mazes with missing walls. Tracing through a 5×5 example by hand revealed the fix, only push the current cell onto the stack after confirming at least one unvisited neighbour exists at that point.

6.2 BFS and DFS Solver

The Grid struct's visited array is shared between the generator and the solver. After maze generation, the recursive backtracker leaves it fully populated with 1 across every cell, so the solver must reset it entirely before running. Getting this wrong causes the solver to treat all cells as already explored and immediately report no solution. The array ultimately uses a three-state encoding (0 = unexplored, 1 = explored, 2 = on solution path), which maps directly to what the renderer needs.

Neither BFS nor DFS inherently records the solution path during traversal, so two parent arrays (`par_x`, `par_y`) were allocated to track the coordinates of its discoverer for each cell. After the end cell is reached, the path is traced backward by following these pointers and marking each cell with state 2. For BFS, a custom circular-buffer queue was also implemented to keep enqueue and dequeue at $O(1)$. DFS by contrast required no new structure at all since LIFO ordering maps directly onto the existing Stack from `maze/stack.h`, using a peek-based approach that mirrors the generator's own traversal.

7. INDIVIDUAL CONTRIBUTIONS

Member	Primary Deliverable	Secondary Contributions	Approx. Effort
Liang Kuan Hon	config/parser.c + parser.h	CLI flag design, malformed-config test suite, README	~25%
Magnus Yunus	maze/generator.c + stack.c/h	Seed RNG testing, DFS stack reuse co-design	~25%
Matthew Phua	solver/bfs.c + dfs.c + queue.c/h	maze_solved.txt format, solver edge-case tests	~25%
Desmond Ngui	render/bmp.c + main.c + grid.h	Integration lead, BMP debugging, Makefile	~25%

7.1 Liang Kuan Hon — Config Parser

Files: `config/parser.c` · `config/parser.h`

Kuan Hon designed and implemented the configuration subsystem. The parser reads an INI-formatted file and populates a shared Config struct used across all modules.

The parser is implemented as an explicit Finite State Machine (FSM) with six states — START, SECTION, KEY, BEFORE_EQ, VALUE, and COMMENT. Each character drives a state transition, enabling robust handling of inline comments, whitespace variations, and files without trailing newlines.

A section-aware dispatch table maps keys to typed handlers, allowing the same key (e.g., algorithm) to be interpreted differently depending on context ([maze] vs [solver]).

The parser performs strict validation, including:

- Type-safe parsing of integers, floats, and strings
- Enum validation for algorithm selection
- Detection of malformed input (missing =, unknown keys, invalid sections)
- Enforcement of structural constraints (minimum dimensions and odd sizing, with auto-correction)

The Config struct serves as the integration contract between modules, ensuring all downstream components operate on validated and consistent data.

7.2 Magnus Yunus — Maze Generator

Files: `maze/generator.c` · `maze/stack.c` / `stack.h`

Magnus implemented the two maze generation algorithms and the shared Stack data structure. The Iterative Recursive Backtracker maintains an explicit integer stack of visited cells. At each step it randomly selects an unvisited neighbour, carves a connecting passage by clearing the

respective wall flags in both cells, and pushes the new cell. When no unvisited neighbours remain it backtracks by popping the stack. Being iterative rather than recursive, it handles 100x100 mazes without any risk of call-stack overflow.

Prim's Algorithm begins from a random seed cell and maintains a frontier list of walls adjacent to the growing maze. A frontier wall is chosen at random, connected to the maze, and its new neighbours added to the frontier. This produces mazes with shorter dead ends and a more uniform branching texture compared to the Backtracker.

- Both algorithms produce perfect mazes: exactly one path between any two cells
- Deterministic output when a seed is provided (`srand(seed)`)
- Custom Stack module reused by the DFS solver — no duplication
- Iterative design prevents call-stack overflow on large grids

7.3 Matthew Phua — BFS & DFS Solver

Files: `solver/bfs.c` · `solver/dfs.c` · `solver/queue.c` / `queue.h`

Matthew implemented both pathfinding solvers and the Queue data structure required by BFS.

The BFS solver uses a custom FIFO queue to explore cells level by level, guaranteeing the shortest solution path in any unweighted maze. Each cell stores a parent index, enabling complete path reconstruction once the exit cell is dequeued.

The DFS solver reuses the Stack from the generator (`solver/dfs.c` links against `maze/stack.c`). It does not guarantee the shortest path but requires less auxiliary memory and can find a valid solution faster in mazes where the exit lies in an early branch.

Both solvers annotate the Grid's visited array and write the reconstructed path to `maze_solved.txt`, giving a text-based ground truth that proved invaluable during debugging.

- BFS guarantees shortest path; DFS trades optimality for reduced memory
- Dynamic Queue grows automatically. No fixed-size overflow possible
- Parent-pointer path reconstruction from exit back to entry
- `maze_solved.txt` output allows text-diff verification independent of the renderer
- `show_visited` config flag exposes explored cells to the renderer for visualisation

7.4 Desmond Ngui — BMP Renderer & Integration

Files: `render/bmp.c` · `main.c` · `grid.h`

Desmond implemented the BMP rendering engine and was responsible for assembling all modules in `main.c`.

The renderer encodes the maze as a 24-bit uncompressed BMP. Wall cells are dark, entrance and exit cells are green, passage cells white, the solution path red, and (optionally) explored cells rendered in a light blue. The BMP file and DIB headers are written bottom-up, byte-by-byte using unsigned char to ensure portability across 32-bit and 64-bit platforms.

A critical requirement of the BMP specification is that each pixel row be padded to a multiple of 4 bytes. The renderer computes `row_size = ((width_px * 3 + 3) / 4) * 4` and writes the

required zero-padding bytes after each row, preventing image corruption on viewers that strictly enforce the specification.

The integration role involved defining `grid.h` as the shared contract, coordinating interface changes with all team members, and resolving the one format mismatch discovered during integration.

- 24-bit uncompressed BMP with correct 54-byte file + DIB header
- 4-byte row padding correct for all maze widths, including odd values
- Optional visited-cell highlighting controlled by the `show_visited` config flag
- `main.c` orchestrates the pipeline with clean memory management and error handling, performing a two-stage config merge, where the INI file is parsed first and CLI flags override afterward.

Effort was distributed equally. All members participated in the kick-off interface design session and the final integration/testing phase. Major design decisions (bit-flag encoding for walls, index encoding for queue entries, BMP cell size parameter) were made collaboratively during weekly meetings.

8. TESTING STRATEGIES

8.1 Maze Generator Tests

The primary correctness criterion for the generator is that it produces a perfect maze: every cell is reachable and there are no loops. This was verified by running the BFS solver on every generated maze and checking that (a) the solver always finds a path, and (b) the number of visited cells equals the total number of cells in the grid, confirming full reachability.

Determinism was tested by running both algorithms with the same seed value and confirming byte-identical BMP output across multiple runs. Seed variation was tested to ensure that different seeds produce visually distinct mazes.

Boundary conditions tested included: a 1×1 maze (single cell, no walls to carve), a 3×3 minimum meaningful maze, a 5×5 maze traced manually against expected output, and a 100×100 large maze to confirm the iterative design does not cause stack overflow. Even-dimension inputs were also tested to verify the odd-dimension enforcement in the parser correctly rejects them before the generator is called.

8.2 BFS and DFS Solver Tests

For the BFS and DFS solvers, testing was done by running the solver against mazes of different sizes and seeds and visually inspecting the `maze_solved.txt` output to confirm that the “.” trail formed a continuous path from start to end. A maze where there is exactly one path between any two cells was also used. Both solvers should always produce the same solution path, which served as a cross-check between the two. Edge cases such as the minimum maze size (5×5) were also tested to ensure the solvers handled small grids without crashing.

8.3 BMP Renderer & Integration Tests

Integration testing focused on the `show_visited` flag, while running the same seed with `show_visited=0` and `show_visited=1`, ensuring the outputs only differ in the light-blue explored cells and the solution path and walls are identical between both renderers.

The full pipeline was also run end-to-end with both solver algorithms to verify the visited state encoding written by BFS and DFS were interpreted consistently by the renderer.

9. LEARNING OUTCOMES

Liang Kuan Hon

Building the FSM parser from scratch, without `sscanf` or `strtok`, gave me a new appreciation for how much complexity hides inside library functions. Designing the 5-state machine forced me to think character-by-character, which is a very different mindset from high-level string processing. I also learned that defining the output interface (Config struct) before writing parsing logic makes integration almost trivial.

Magnus Yunus

Working on the maze generator showed me how much the choice of algorithm shapes the output, the Backtracker produces long winding corridors while Prim's gives shorter dead ends and a more uniform texture. Writing both iteratively using an explicit stack also taught me the practical value of avoiding recursion for large inputs, the state stays visible and debuggable. Having the Stack module reused by the DFS solver was a nice bonus that reinforced how a clean interface pays off at integration time.

Matthew Phua

Building the BFS and DFS solvers made it clear that the choice of data structure is what drives the algorithm. If we use a queue then we get BFS, swap it for a stack and we get DFS. Since no external libraries were allowed, the queue had to be built from scratch and path reconstruction required an extra pair of parent arrays to trace the solution back from the end to the start. Working in a shared codebase also taught me to be careful about module boundaries especially where the `grid->visited` array is written by the generator and then reused by the solver. Forgetting to reset it before solving would silently break everything without any obvious error.

Desmond Ngui

Writing the BMP renderer from scratch forced me to understand the format at the byte level, such as handling 14-byte file headers and 40-byte DIB headers. While debugging the row-padding issues, I have learnt to reach for the hex editor whenever a binary format misbehaves to inspect the actual byte-level alignment, rather than adding print statements that cannot tell anything useful about the raw byte layout.

9.1 Team-Wide Takeaway

Interface-first design is the practical difference between parallel development and serial development. Freezing `grid.h` and the Config struct initially meant all four modules could be written concurrently and integrated in focused sessions rather than being blocked waiting for each other. This habit will carry forward into all future group projects.